03/09/24

Advance Algorithm (PB)

1) All pair shortest path Problem — $O(n^3)$ n = vertex

2) 0-1 Knapsack problem

Dijkstra shortest path $O(|E| \log |V|$

Input: Given weighted directed graph

Assumption: graph as adjacency matrix

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ w(i,j) & \text{if } i \neq j \text{ and } (i,j) \in E, \text{edge} \\ +\infty & \text{if } i \neq j \text{ and } (i,j) \notin E, \text{no edge} \end{cases}$$

$n \times n$ matrix

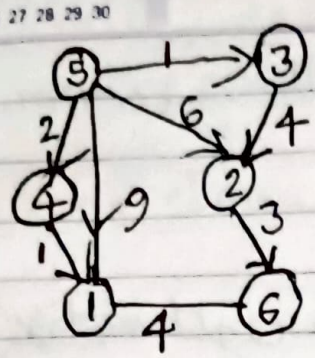
# of vertices

① characterizing the problem

$D_{n \times n} = d[i,j] =$ i to j shortest path

d_{ij}: shortest path i to j

such that any intermediate values on the path are chosen the set $\{1 \ldots k\}$



$d_{5,6} = \alpha$

$d^{(1)}_{56} = 13$

$d^{(2)}_{56} = \min\{13, 9\} = 9 \langle 5,2,6 \rangle$

$d^{(3)}_{56} = \min\{13, 9, 8\} = 8 \langle 5,3,2,6 \rangle$

$d^{(4)}_{5,6} = \min\{13, 9, 8, 6\} = 6 \langle 5,4,1,6 \rangle$

2) formulation of recurrence of DP

i) Take the kth vertex: take shortest path $d^{(k-1)}_{i,k}$ + $d^{(k-1)}_{kj}$ ie the shortest path from i to k and k to j taking values $\{1 \ldots k-1\}$

ii) Do not take the kth value: shortest Path from i to j uses only values from $\{1 \ldots k-1\}$ and its length is d_{ij}

$d_{ij} = w_{ij}$, intialization $k=0$

$d^{k}_{ij} = \{ \min(d^{(k-1)}_{ij} + d^{(k-1)}_{kj}, d^{(k-1)}_{ij})$ for $k \geq 1$

iii) computing the shortest path in bottom up fashion.

```
Floyd warshall (n, w) {        -> weight matrix
                               -> # of vertices
    d[1...n][1...n], mid[1...n][1...n]
                               -> whether kth vertex is chosen or not
    for i 1 to n
        for j 1 to n
            d[i,j] = w[i,j]
            mid[i,j] = null
        }
    }
    for k=1 to n {
        for i=1 to n {
            for j 1 to n {
                if d[i,k]+d[k,j] < d[i,j] {
                    d[i,j] = d[i,k]+d[k,j]
                    mid[i,j] = k
                }
            }
        }
    }
    return d, mid
```

I) value of shortest path

II) K is chosen or not in mid

4) Extraction of shortest path using mid matrix.

```
Extract Path(i, j, mid) {
    if mid[i,j] = null
        return(i, j)
    else
        Extract path (i, mid[i,j])
        Extract path (mid[i,j], j)
```

0-1 Knapsack problem

Input: Given a set of n objects with cost/value & weight w_i, and the capacity W of a knapsack, $i = 1$ to n

SUNDAY 28

Object: Determine the subset $T \subseteq \{1, 2, \ldots, n\}$ that maximizes the profit (value/cost

maximize $\sum_{i \in T} v_i$ subject to

$$\sum_{i \in I} w_i \leq W$$

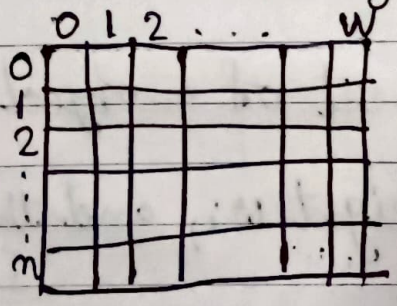
$V[i, j] \rightarrow$ capacity of knapsack
$\hookrightarrow$ Objects

$W = 9$ $j = 1$ to 9

V is a matrix with size $0 \ldots n$ and $0 \ldots W$

where $1 \leq i \leq n$, $0 \leq j \leq W$

$V[i, j]$ = max value of any subset of ~~of~~ objects 1 to i that can fit in a knapsack of capacity / weight j



max profit subject to contain z of w $\Rightarrow V[n, W] \rightarrow$ final optimal profit

$V[0, j] = 0$ → no object chosen

$V[i, 0] = 0$ → max weight is 0

Leave object : $V[i-1, j]$: profit remains as before as ith object is not chosen

Take object i : $V[i-1, \underset{j - w_i}{?}] + V_i$

Recurrence relation:

$$V[0, j] = 0$$

$$V[i, j] = \begin{cases} V[i-1, j] & \text{if } w_i > j \\ \max \begin{cases} V[i-1, j], \\ V[i-1, j-w] + V_i \end{cases} & \text{if } w_i \leq j \end{cases}$$

<u>bottom up solution</u>

~~matrix V~~

fill up matrix V row wise

Knapsack ($\overset{\text{value of object}}{V[1 \ldots n]}$, $\underset{\text{weight of object}}{w[1 \ldots n]}$, $\overset{\text{no of knapsack}}{n}$, $\overset{\text{capacity of knapsack}}{W}$)

$V[0 \ldots n, 0 \ldots W]$

for $j = 0$ to W

$V[0, j] = 0$

for $i = 1$ to n {

for $j = 0$ to W

if $j >= w[i]$

take object $= v[i] + V[i-1, j-w[i]]$

else

take object $= -\infty$ //cant take i

$V[i,j] = \max\{$ leave object, take object $\}$

-> mid$[i,j] = i$ //extract which item the object in optimal soln

return $V[n,W]$

$O(nW)$